Near-Frontier with Reset (NFR) Planner: Formal Overview

ARC-AGI Abstraction Navigator Notes

November 9, 2025

1 Problem Setting

Let $\mathcal{G} = (S, A, T)$ denote the agent interaction graph. States $s \in S$ encode masked ARC frames, actions $a \in A$ are constrained to a caller-provided navigational action set $A_{\text{nav}} \subseteq A$ (typically the four arrows; the special action RESET is treated separately), and $T : S \times A \to S \cup \{\perp\}$ is the partial transition function estimated from experience ($\perp$ indicates an unknown successor).

Each play starts at a known level start state $s_0 \in S$. The navigator also tracks the current level index ℓ_t and stores the terminal state $s_{\text{term}}(\ell)$ whenever a level completion is observed. The planner may be given an optional goal hash g (for example, the current level's recorded terminal state) to prioritise reaching it before resuming generic exploration. The planner is given:

- $S_{\text{disc}} \subseteq S$: discovered states from prior episodes.
- $T_{\text{known}} : S_{\text{disc}} \times A \to S_{\text{disc}} \cup \{\perp\}$: the partial transition map recovered from experience (returning $\perp$ when a transition has not yet been seen).
- $A(s) \subseteq A$: actions the environment currently allows at state s.
- $S_{\text{fail}} \subseteq S_{\text{disc}}$: recorded failure states (e.g., game-over screens) that should not be expanded.
- $\mathcal{F} = \{\, s \in S_{\text{disc}} \setminus S_{\text{fail}} \mid \exists a \in A_{\text{nav}} : T_{\text{known}}(s, a) = \perp \,\}$, the *frontier*: discovered states with at least one action in A_{nav} whose successor is unknown in T_{known}.

Failure states are updated whenever a transition produces a game-over screen, ensuring the planner does not repeatedly probe known dead ends.

During an episode the agent sits in state s_t with available actions $A_t = A(s_t)$. The planner must output $a_t \in A_t \cup \{\text{RESET}\}$ to minimise the distance to a frontier while optionally using reset jumps to s_0. When a terminal state $s_{\text{term}}(\ell)$ is known for the current level and supplied as `target_state`, that state is treated as a temporary goal that can outweigh frontier expansion.

2 Level-Aware Objectives

Before evaluating frontier costs, the navigator performs quick goal checks:

1. **Current-level replay.** If $s_{\text{term}}(\ell_t)$ is known, the caller supplies `target_state` $= s_{\text{term}}(\ell_t)$ to reproduce the previously discovered solution path.
2. **Optional external goal.** If the caller supplies a different goal hash (e.g., a higher-level terminal), the same mechanism steers play toward it until it is reached, after which the planner reverts to generic frontier minimisation.

Both checks reuse the distance maps defined below, so the extra computation is negligible.

3 Algorithm

The planner combines goal-directed replay with frontier exploration. At every decision point it performs the following sequence:

1. **Objective selection.** If a goal state g has been supplied, that state becomes the temporary objective. Otherwise the goal set is the frontier $\mathcal{F}$.

2. **Distance computation.** Run breadth-first search on the discovered subgraph $\mathcal{G}_t = (S_{\text{disc}}, A, T_{\text{known}})$ from both the current state s_t and the level start s_0:

$$D_t^c, P_t^c = \text{BFS}(\mathcal{G}_t, s_t), \tag{1}$$
$$D_t^0, P_t^0 = \text{BFS}(\mathcal{G}_t, s_0). \tag{2}$$

   These maps provide shortest-path distances and predecessors inside the known subgraph. Both traversals consider only A_{nav} edges.

3. **Candidate evaluation.** If a specific goal g was chosen, evaluate it directly: when g is reachable from s_t (i.e. $g \in \text{dom}(D_t^c)$), follow the path recovered from P_t^c; otherwise ignore g and proceed with frontier selection. When no goal is set, compute the reset-aware cost for every frontier node as

$$J(f) = \min\{D_t^c(f), 1 + D_t^0(f)\}. \tag{3}$$

   Select the candidate with lexicographic order on (J, D_t^0, f), breaking ties on (J, D_t^0) by the canonical frame-hash identifier.

4. **Action selection.** If the chosen candidate is reachable and A_t is non-empty, follow the first edge on the path recovered from P_t^c when D_t^c is no greater than $1 + D_t^0$. Otherwise return RESET, indicating that a jump to s_0 plus replay from there is cheaper. When already positioned at the chosen frontier (i.e., the recovered path is empty), execute the first available action in A_{nav} whose successor is unknown; if none exists, the planner abstains by returning `None`. When no candidate has finite cost, the planner abstains.

The resulting policy naturally alternates between fast replay of known solutions (when a goal state is provided) and systematic exploration of the nearest frontier.

4 Desirable Properties

Completeness on the Known Subgraph. As long as $\mathcal{G}_t$ is connected between s_t and all frontiers, BFS ensures $D_t^c(f)$ equals the true shortest distance in $\mathcal{G}_t$. Therefore the navigation step reaches $f^\star$ in minimal steps without reset.

Reset Optimality. The cost (3) chooses between continuing from s_t or incurring one reset plus the path from s_0. This is optimal under the assumption that resetting teleports to s_0 in one step and costs a unit time penalty.

Frontier Prioritisation. Any frontier with strictly smaller $J(f)$ than all others will eventually be chosen. If exploration uncovers new edges, J automatically updates without retuning.

Level Progression. Supplying a terminal hash as `target_state` enables deterministic replay from the current state when the goal is reachable in the known subgraph; otherwise the planner reverts to frontier search. Reset-versus-continue trade-offs are applied during frontier navigation, not during goal replay.

Idempotence of Known Actions. The planner never replays an action whose successor is already in $T_{\text{known}}(s_t, a)$ when probing frontiers; thus exploration focuses on unknown transitions.